Lesson 3: Take Your Shot

Unity

VIDEO GAME CREATION

This week: spawning objects at runtime. Your job is to make your player shoot a projectile that flies up the screen and cleans itself up when it leaves - the first piece of your capstone's actual shooting mechanic.

Open your own [YourName]Game project from Lesson 2 - don't start a new one. Everything this week builds on the player object you already have.

Tip: this lesson's Assets folder has a ready-to-use placeholder projectile image (bullet_yellow.png) and a new sound effect (sfx_shoot.wav) - you don't need to find or make your own art today.

Setup

1. Open your [YourName]Game project from Lesson 2.
2. Create a new empty GameObject, add a Sprite Renderer component, and assign bullet_yellow.png (import it from this lesson's Assets folder into your Project window first) as its Sprite.
3. Drag this GameObject from the Hierarchy into the Project window to turn it into a Prefab, named Bullet - then delete the one left in the scene (the Prefab asset is all you need going forward).
4. Create a new C# script named Bullet, attach it to the Bullet prefab.

Level 1 - Minimal (MVP)

Pressing the fire key spawns one projectile that flies upward and disappears when it leaves the screen. This is a complete finish for today - reaching this is enough to move on to next week.

5. In the Bullet script, add a public float speed and an Update() that moves it upward and destroys it once it's off-screen (code below).
6. On the Player script (from Lesson 2), add a public GameObject bulletPrefab field and check Keyboard.current.spaceKey.wasPressedThisFrame in Update() to Instantiate() it at the player's position.
7. In the Inspector, drag the Bullet prefab from the Project window onto the Player's Bullet Prefab field.
8. Press Play and hit Space to test - a bullet should spawn at the player and fly upward, then disappear.

```
// Bullet.cs, attached to the Bullet prefab:
public float speed = 6f;
```

```
void Update() {
    transform.Translate(Vector3.up * speed * Time.deltaTime);
    if (transform.position.y > 6f) Destroy(gameObject);
}

// Player.cs - add a field and a check inside Update():
public GameObject bulletPrefab;

if (Keyboard.current.spaceKey.wasPressedThisFrame) {
    Instantiate(bulletPrefab, transform.position, Quaternion.identity);
}
```

Alternate Path: Visual Scripting

The same spawn/move/cleanup pattern works in a Script Graph instead of C# - Unity's Visual Scripting graphs compile to the same underlying calls, using the same new Input System as the code path.

9. Open PlayerControls (the Input Actions asset from Lesson 2's Player Input component) and add a new action to the Player action map named Fire, Action Type Button, bound to Space. Click Save Asset.
10. On the Bullet prefab, add a Script Machine component and open a new Script Graph.
11. Add an Update event node, a "Translate" node (Transform) feeding it Vector3.up * speed * deltaTime, and a Branch node checking the Y component of the position against your off-screen threshold, feeding a "Destroy Game Object" node on True.
12. On the Player's own graph, right-click and add an "On Input System Event Button" node. Leave Target as This, set Input Action to Fire, and set Input Action Change Type to On Pressed. Wire its Trigger output into an "Instantiate" node (with the Bullet prefab as its asset input).

Level 2/3: an On Input System Event Button node set to On Pressed already only fires once per physical press (event-driven, not polled), so no extra cooldown logic is required for Level 1/2. Level 3's ideas are all reachable with more nodes: an Audio Source node for sound, a "Set Member" node on the Sprite Renderer's color for the visual idea - same ideas as the C# path, just wired instead of typed.

Level 2 - Complete (Alpha/Beta)

Firing repeatedly produces multiple independent projectiles on screen at once, each moving and disappearing correctly, and rapid-fire is capped so holding the key down doesn't flood the screen.

- Test firing several times quickly - each press should produce its own independent bullet instance, all moving and destroying themselves correctly. Since each Instantiate() call creates a fully separate GameObject, this already works with no extra code.
- Add a cooldown so holding Space doesn't fire every frame: wasPressedThisFrame already only returns true on the single frame a key is first pressed (not every frame held), so Level 1 is already immune to this specific bug - add a private float fireCooldown = 0f timer anyway if you want a slower, tunable fire rate (count it down by Time.deltaTime each frame, and only allow firing when it's <= 0, resetting it to e.g. 0.2f on each shot).

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: add an AudioSource component to the Player, assign sfx_shoot.wav as its AudioClip, and call audioSrc.Play() right next to the Instantiate() call.
- Visual: give the Bullet prefab its own SpriteRenderer.color or a small scale-up pulse that fades back to normal after a fraction of a second, so it reads clearly as "a shot" rather than a copy of the Player - not a permanent recolor.
- Structure: keep everything about how a bullet behaves - its own movement, its own off-screen cleanup - inside Bullet.cs, and keep Player.cs limited to just spawning it (Instantiate + cooldown + sound). Don't reach into a bullet's internals from the Player script, or vice versa.

If Something's Wrong

- Nothing happens on Space: confirm the Bullet Prefab field on the Player is actually assigned in the Inspector - a null reference here throws an error in the Console instead of silently failing.
- Bullet spawns but doesn't move or never disappears: confirm Bullet.cs is attached to the Bullet prefab itself (check the Project window, not just the scene), and that Update() is spelled exactly right (case-sensitive).
- Bullets pile up forever: double-check the y > 6f threshold roughly matches your camera's visible area - a Size-5 orthographic camera shows about -4.5 to 4.5 vertically, so 6f is safely just off-screen.

Want More?

Student Resource List > Unity section, or learn.unity.com